

Problem 1: Return all possible permutations of a string or array elements.

```
def permutations(text):  
  
    if len(text) == 1:  
        return [text]  
  
    result = []  
    for i in range(len(text)):  
        current_char = text[i]  
  
        remaining = ""  
        for j in range(len(text)):  
            if j != i:  
                remaining += text[j]  
  
        sub_permutations = permutations(remaining)  
  
        for perm in sub_permutations:  
            result.append(current_char + perm)  
  
    return result
```

Problem 2: Fill a 9x9 Sudoku board following Sudoku rules.

```
def is_valid(arr, row, col, num):  
    for c in range(9):  
        if arr[row][c] == num:  
            return False  
  
    for r in range(9):  
        if arr[r][col] == num:  
            return False  
  
    startRow, startCol = 3 * (row // 3), 3 * (col // 3)  
    for r in range(startRow, startRow + 3):  
        for c in range(startCol, startCol + 3):  
            if arr[r][c] == num:  
                return False  
  
    return True  
  
def sudoku(arr):  
    for row in range(9):  
        for col in range(9):  
            if arr[row][col] == 0:  
                for num in range(1, 10):  
                    if is_valid(arr, row, col, num):
```

```

        arr[row][col] = num
        if sudoku(arr):
            return True
        arr[row][col] = 0
    return False
return True

# Problem 3: Partition a string such that every substring is a palindrome. Return
all possible partitions.
    # i used some help from chatgpt because it was hard to understand
def is_palindrome(text):
    return text == text[::-1]
def palindrome_partitions(text):
    if not text:
        return [[]]

    result = []

    for i in range(1, len(text)+1):
        prefix = text[:i]
        if is_palindrome(prefix):
            for partition in palindrome_partitions(text[i:]):
                result.append([prefix] + partition)

    return result

```